

ArkLib’s KZG evaluation-binding theorem is axiom-clean **and** carries no information at any parameter: its `t-SDH` assumption quantifies over an unrestricted adversary type, which a `Classical.choice` trapdoor extractor inhabits with success probability exactly 1. We mechanize the refutation, the extraction-shaped repair that provably survives the exact attack, and the generic-group security bound — now mechanized **end to end, in both standard generic-group models**: Maurer (explicit equality) wired to ArkLib’s real `tSdhExperiment`, Shoup (random encoding) standalone at the identical bound, side-conditions named per model.

Research note — internal, not filed, not a security advisory: a **formalization-soundness** issue in a public, in-development library, not a vulnerability in any deployed system. KZG, `t-SDH`, and the reduction are sound as normally stated — the issue is a Lean quantifier.

I · The finding: axiom-clean, and empty

The mechanism

KZG evaluation binding is proved in ArkLib by a correct, constructive reduction to the `t-SDH` assumption — whose adversary is a **plain, unrestricted function type**:

```
abbrev tSdhAdversary (D : ℕ) :=
  Vector G₁ (D+1) × Vector G₂ 2 → StateT unifSpec.QueryCache ProbComp (Option (ZMod p × G₁))

def tSdhAssumption (D : ℕ) (error : ℝ≥0) : Prop :=
  ∀ (adversary : tSdhAdversary D), tSdhExperiment D adversary ≤ error
```

`ProbComp` is a free monad over oracle queries: only `query` nodes cost anything; pure computation is free and no resource bound is imposed anywhere. The SRS includes the verifier `leg` (g_2, g_2^τ), which determines τ whenever $g_2 \neq 1$, and ArkLib’s own `exists_zmod_power_of_generator` (`Algebra.lean:105`) makes the discrete log `Classical.choice`-definable. The adversary reads g_2^τ , recovers τ , and returns the `t-SDH` solution ($c = 0, g_1^{1/\tau}$) — zero oracle queries, success probability exactly 1.

```
noncomputable def tauExtractingAdversary (hg₂ : g₂ ≠ 1) (D : ℕ) : tSdhAdversary D :=
  fun srs => pure (some (0, g₁ ^ (1 / dlog0f hg₂ srs.2[1])).val))

theorem tSdhExperiment_tauExtractingAdversary : tSdhExperiment D (...) = 1
theorem not_tSdhAssumption (herr : error < 1) : ¬ tSdhAssumption D error
theorem tSdhAssumption_trivial_of_one_le (1 ≤ error) : tSdhAssumption D error
```

No content at any parameter

Below 1 the assumption is refuted; at ≥ 1 its conclusion is the triviality “a probability is ≤ 1 .” `binding` consumes the assumption in the last step of its `calc`, and its hypothesis `hpair : pairing g₁ g₂ ≠ 0` **forces** $g_2 \neq 1$ (a bilinear map kills the identity) — exactly what the extractor needs. So `binding`’s hypotheses are jointly unsatisfiable for any error below 1 (`binding_hypotheses_unsatisfiable`), and its conclusion is free at ≥ 1 . The sibling `ARSDH` assumption falls identically, taking `function_binding` with it. A canary — the giving-up adversary scores exactly 0 — confirms the experiment discriminates. The reduction itself is fully constructive and algebraically correct; the vacuity lives entirely in the assumption’s quantifier.

The pattern, not a typo

Reducing to a **different** base assumption does not escape. The natural `q-DLOG` assumption, stated in ArkLib’s own unrestricted-adversary idiom, is equally false below 1 (`not_qDlogAssumption`, same extraction, own canary). And ArkLib’s algebraic-group-model scaffolding confirms the disease from the other side: `AGM/Basic.lean` is a stub (its adversary’s `run` is literally `sorry`; zero theorems; orphaned) and is **unsound as written** — the adversary is a `ReaderT` over the concrete group table, so its outputs can still depend on discrete logs. Its author flags the open problem verbatim: “TODO: need to be sure this definition is correct” “How to make the adversary truly independent of the group description? It could have had `G` hardwired.” Any concrete-group assumption of the shape $\forall$ unrestricted adversary, $\Pr[\text{win}] \leq \varepsilon < 1$ is `Classical.choice`-false in this idiom.

The methodological point: axiom checks are blind to vacuity

`binding`, the refutation, and the winning adversary all print the **same** clean axiom closure:

```
#print axioms not_tSdhAssumption -- [propext, Classical.choice, Quot.sound]
```

No `sorryAx`, nothing any `#print axioms / #assert axioms` gate would flag. **Axiom-clean and vacuous coexist**. The blindness is structural: an axiom check reports a proof term’s closure; it never asks whether the theorem’s **hypotheses** are jointly satisfiable, and any `def FooHard : Prop` used as a hypothesis is an assumption no axiom check inspects. Query-based bounds (`IsQueryBoundP`) do not help either — the winning adversary makes zero queries. The only reliable test is adversarial: try to **inhabit the assumption’s negation** — prove the floor false at its deployed parameters. We found the identical pattern in our own hardness floors first, in several places, before ever looking at ArkLib; we present this as a field lesson, not a dunk.

II · The repair, the number — both standard GGM models, mechanized

The mergeable de-vacuation (mechanized)

ArkLib’s reduction is already constructive and the assumption is consumed at exactly one `calc` step — so split there: the unconditional prefix becomes the primary theorem, the original binding a one-line corollary.

```
theorem binding_reduces_to_tSdh ... (adversary : KzgBindingAdversary ...) :
  bindingExperiment ... adversary ≤ tSdhExperiment g₁ g₂ n (bindingReduction ... adversary)
```

No assumption `Prop`: both sides are concrete probabilities, true at every parameter — nothing for `Classical.choice` to inhabit (+41/−14 in one file; whole tree builds, 2994 jobs; axiom-clean). It **provably survives the exact attack** (`repair_survives_attack`): in one `sorry-free` closure, the trapdoor adversary still refutes `tSdhAssumption` below 1 **and** the repaired bound holds regardless — premise removed, reduction kept, the one obligation a sound assumption class must discharge isolated.

The number: the generic bilinear group bound

Group elements become opaque handles carrying **ordinary** polynomials in $\mathbb{Z}_p[X]$ — **not Laurent**: group inversion negates the exponent, it never introduces X^{-1} . A winning $1/(X + c)$ is therefore unrepresentable, and a “win” forces the nonzero, degree- $\leq D + 1$ polynomial $F_\ell \cdot (X + c)$ — 1 to vanish at the random τ . Simulation (identical-until-bad) plus Schwartz–Zippel yield Boneh–Boyen’s Theorem 12, verified line by line against the source:

$$\varepsilon \leq (q_G + D + 3)^2 \frac{D + 1}{p - 1} = O\big((q_G + D)^2 \cdot D/p\big)$$

Not a clean q^2/p : the bound is cubic in the SRS degree D , and at production parameters the D^3/p term is the one to watch (Corollary 13’s $q < O(p^{1/3})$ side condition). Naive AGM is no shortcut — a `Classical.choice` adversary returns a **valid** representation too (validity is not independence); it relocates the same content onto `q-DLog`’s generic hardness.

Both standard GGM models, end to end (mechanized, sorry-free, no sorryAx)

The two standard formalizations of “the adversary cannot see group elements” differ only in **how it learns equalities of held handles**; both are mechanized at the identical bound, a genuine < 1 when $\binom{\text{fuel}+D+4}{2} \cdot D + (D + 1) < p - 1$.

Maurer (explicit equality) — wired to ArkLib. The capstone `GgmEndToEnd.tSdh_ggm_sound` is about ArkLib’s **own Groups.tSdhExperiment**, restated nowhere — for every generic strategy `strat` and query budget `fuel`:

```
tSdh_ggm_sound : tSdhExperiment D (embed strat) ≤ (C(fuel+D+4, 2)·D + (D+1)) / (p - 1)
```

Equality costs an explicit query (`Move.query`), so only queried pairs can collide — the all-pairs count on the right is a sound **over-count** here. **Why it escapes the vacuity**: it quantifies not over the full `tSdhAdversary` type — provably **false** there (Panel I) — but over the **image of the generic embedding** `embed`: a strategy receives only equality booleans, never a group element, so it realizes only $g_1^{f(\tau)}$ with $\deg f \leq D$. **Side-conditions, named**: $1 \leq D$ (genuinely **false** at $D = 0$: with no pairing, a G_1 adversary cannot form g_1^τ); $2 \leq p$; `orderOf g₁ = p` (with $g_1, g_2 \neq 1$); ArkLib’s own `SampleableType` instance.

Shoup (random encoding) — standalone. `GgmShoup.shoup_ggm_sound`: the adversary sees random encodings under an injection $\sigma : \mathbb{Z}_p \hookrightarrow E$ and compares **all** held pairs **for free** — no query move, the full equality matrix (`eqPattern`) at every step — so every held pair is a live collision candidate and the **same** all-pairs count is **tight**. The matrix-valued identical-until-bad (`runShoup_congr_off_bad`) is **proven**, degree invariants discharged; σ never enters (injectivity folds it away, as $a \mapsto g_1^a$ does in the Maurer `embed`). **Side-conditions**: $1 \leq D, 2 \leq p$, `Fact (Nat.Prime p)` — nothing else (it never touches the group experiment, so no generator or `SampleableType`). Wiring Shoup into ArkLib would be optional and redundant — Maurer is the wired track. Earlier versions of this spread labelled the wired capstone “Shoup random-encoding”: that name belongs here, to the standalone theorem — the capstone is Maurer. `#print axioms` on both, full spines: `[propext, Classical.choice, Quot.sound]`.

The mechanized spine (all sorry-free, axioms [propext, Classical.choice, Quot.sound])

- **Vacuity refutations** — `t-SDH`, `ARSDH`, `q-DLOG`, each with a discriminating canary, against genuine ArkLib; the repair and its survival, `binding_reduces_to_tSdh / repair_survives_attack`.
- **The counting bounds** — static core `ggm_tSdh_sound`: $\varepsilon \leq (D + 1)/(p - 1)$ over the **entire** committed-generic type (the trapdoor extractor is untypable here; as far as our census found, the first generic-group security theorem in Lean); Maurer explicit-equality `adaptive_ggm_sound` (identical-until-bad **proven by induction**, not assumed); the shared all-pairs counting core `rand_encoding_bound` (bad event and table size are theorems; over-count in Maurer, tight in Shoup; its $\delta = D$ specialization feeds both).
- **Degree discharge + wiring, on the ACTUAL oracle** — ArkLib’s `tSdhAdversary` is granted **no pairing map**, so the oracle is purely linear: `hdeg_out_of_run / hdeg_handles_of_run` prove $\text{natDegree} \leq D$ by induction on the **real** `runAux/runTable` recursion — the $\delta = D$ bound hypothesis-free. `embed_run_correspondence` (the group `run` in lockstep with the symbolic `run`); `experiment_eq_count` (ArkLib’s game collapses to $(\text{winSet.card})/(p - 1)$); `groupWinSet_eq_realWinSet` (the win predicate is ArkLib’s real `tSdhCondition`, by prime-order injectivity).
- **Off the critical path** (optional — gates nothing) — the pairing-aware $\delta = 2D$ ceiling (`degree_invariant_paired`; naive flat $2D$ claim **refuted**, X^4 at $D = 1$); re-typing `bindingReduction` as a `Strat` — the bound already covers the whole `embed` image.